SAPIENZA
UNIVERSITÀ DI ROMA

“SAPIENZA” UNIVERSITY OF ROME

FACULTY OF INFORMATION ENGINEERING,
COMPUTER SCIENCE AND STATISTICS

DEPARTMENT OF COMPUTER, CONTROL
AND MANAGEMENT ENGINEERING

MASTER’S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

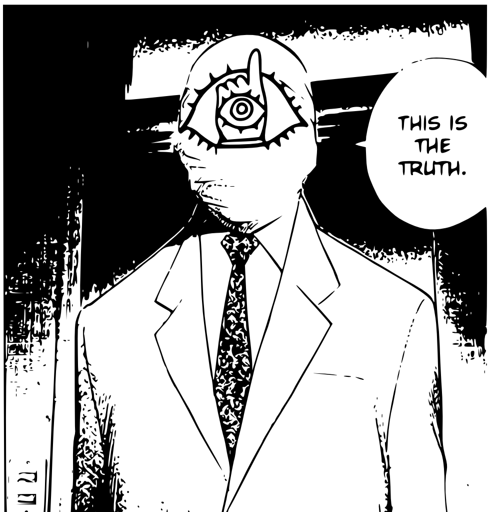
Reinforcement Learning

Lecture notes

Author
Marco Casu
https://github.com/CasuFrost

October 19, 2026

This document summarizes and presents the topics for the Reinforcement Learningcourse for the Master's degree in Artificial Intelligence and Robotics at Sapienza University of Rome. The document is free for any use. If the reader notices any typos, they are kindly requested to report them to the author.


THIS IS THE TRUTH.

CONTENTS

1 Exploration and Exploitation **3**
- 1.1 Multi-Armed Bandits . . . 3
  - 1.1.1 The Hoeffding Inequality and Regret Calculation . . . 4
- 1.2 Contextual and Bayesian Bandits . . . 8
  - 1.2.1 Explore and Commit . . . 9
  - 1.2.2 ϵ-Greedy . . . 10
  - 1.2.3 Gaussian Bayesian Bandits . . . 10

2 Planning in MDPs **12**
- 2.1 Markov Decision Processes . . . 12
  - 2.1.1 MDP Horizon . . . 14
- 2.2 Optimality of a Policy . . . 16
- 2.3 Value iteration . . . 19
  - 2.3.1 Exact Policy Evaluation . . . 19
  - 2.3.2 Fixed Point Iteration . . . 20
- 2.4 Policy Iteration . . . 25
  - 2.4.1 Performance Difference . . . 26
- 2.5 Finite-Horizon MDPs . . . 27

3 Control Theory and Reinforcement Learning **29**
- 3.1 Linear Quadratic Regulator . . . 30
  - 3.1.1 Extensions . . . 31
- 3.2 LQR for Non-Linear Systems . . . 32
  - 3.2.1 Trajectory Tracking . . . 33
- 3.3 Iterative LQR . . . 33

CHAPTER

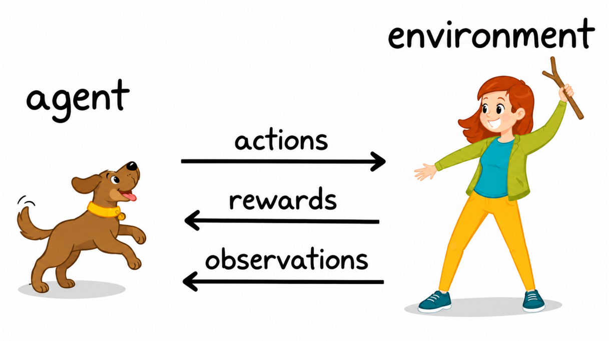
1

EXPLORATION AND EXPLOITATION

Definition 1.0.1 : Reinforcement Learning

With **Reinforcement Learning** we mean the process to *learn* what to do in order to maximize a numerical reward signal:

- The learner must discover which actions yield the highest reward by trying them.
- The actions might affect not only immediate reward, but also subsequent ones. It is a *trial-and-error* process.



The agent has to:

- **Exploit** what it has already experienced to obtain a reward.
- **Explore** in order to make better action selections in the future.

One of the main challenges of reinforcement learning is the exploration-exploitation trade-off. In stochastic tasks, each action must be tried many times, in order to correctly estimate the expected reward.

1.1 Multi-Armed Bandits

One of the main problems in reinforcement learning is to decide when we should continue to explore (collect more information about the environment) or to exploit (use the available information in order to get closer to the goal). We need to *carefully* and *systematically* explore. The **multi-armed bandits (MAB)** problem is a simplified problem that describes the exploration paradigm.

We have K different arms, that are our actions:

$$a_1, a_2, \dots, a_K \tag{1.1}$$

and a reward $r \in [0, 1]$, that is the scalar signal representing a feedback. Each single arm has an unknown reward distribution ν_i with mean μ_i, every time we pull one of the K arms we observe a reward. We interact with the MAB in the following way:

- for $t = 0, 1, \dots, T-1$:
    1. pull an arm $I_t \in \{a_1, a_2, \dots, a_K\}$ based on historical information
    2. observe the reward $r_t \sim \nu_{I_t}$ of the arm I_t.

The goal of this process is to minimize the **opportunity loss**. Assume that the best arm to pull is known (a^*), with its mean reward (μ^*):

$$\mu^* = \max_{i \in \{1,\dots,K\}} \mu_i. \tag{1.2}$$

Consider the total expected reward obtained if we pull the best arm for T rounds:

$$T\mu^* \tag{1.3}$$

and the total expected reward of the arms we actually pulled over T rounds:

$$\sum_{t=0}^{T-1} \mu_{I_t} \tag{1.4}$$

the **regret** is the difference between these two quantities:

$$\text{Regret}_T = T\mu^* - \sum_{t=0}^{T-1} \mu_{I_t} \tag{1.5}$$

the regret is the expression of the opportunity loss, which we want to minimize. In the MAB example, the exploration-exploitation trade-off manifests itself in the form of the following question:

> *Should we pull arms that have been tried less frequently in the past (i.e., explore), or should we commit to the current best arm (i.e., exploit)?*

We consider different strategies. The **greedy algorithm** tries each arm once, and then commits to the one that has the highest observed reward. This algorithm is not optimal: a low mean arm might generate a high reward the single time we sample it.

1.1.1 The Hoeffding Inequality and Regret Calculation

A better choice of algorithm is the **Explore & Commit**: we try each arm multiple times, and we compute the empirical mean of the rewards. Then we commit to the arm with the highest empirical mean.

1. Let N be a fixed value, and $T \geq NK$
2. for $k = 1, \dots, K$
    - pull the arm a_k for N times
    - observe the set of observations $\{r_1^{(k)}, r_2^{(k)}, \dots, r_N^{(k)}\}$
    - compute the empirical mean $\hat{\mu}_k = \frac{1}{N}\sum_{i=1}^{N} r_i^{(k)}$
3. for $t = NK, \dots, T-1$

- pull the best empirical arm $I_t = \underset{i \in \{1,\ldots,K\}}{\arg\max}\, \hat{\mu}_i$

Proposition 1.1.1 : Hoeffding Inequality

Given a distribution with mean μ, where we have observed N independent and bounded samples $r_i \in [0, 1]$, the following inequality holds with probability $1 - \delta$:

$$\left| \frac{1}{N} \sum_{i=1}^{N} r_i - \mu \right| \leq \sqrt{\frac{\ln(2/\delta)}{2N}}. \tag{1.6}$$

The Proposition 1.1.1 gives us a confidence interval on our empirical mean.

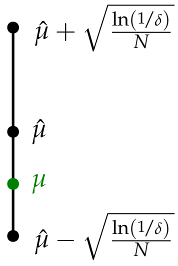

$\hat{\mu} + \sqrt{\frac{\ln(1/\delta)}{N}}$
$\hat{\mu}$
μ
$\hat{\mu} - \sqrt{\frac{\ln(1/\delta)}{N}}$

Figure 1.1: The true mean is located at a bounded distance from the sampled mean with probability $1 - \delta$.

For example, if $\delta = 0.01$, the bound holds with probability 99%. The Hoeffding inequality can also be applied for all the arms at once. During the exploration phase, for **all the arms**: with probability $1 - \delta$ the following inequalities hold:

$$\left| \frac{1}{N} \sum_{i=1}^{N} r_i^{(k)} - \mu_k \right| \leq \sqrt{\frac{\ln(2K/\delta)}{2N}} \quad \text{for every } k \in \{1, \ldots, K\}. \tag{1.7}$$

This comes from a union bound over the K arms, which is why $2/\delta$ becomes $\frac{2K}{\delta}$.

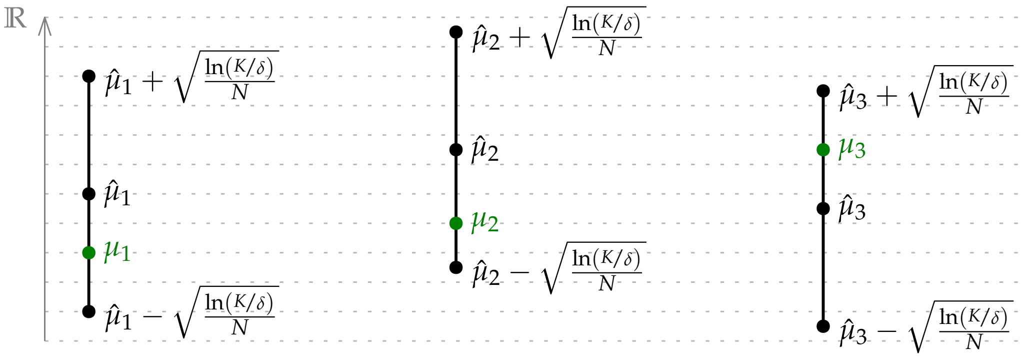

$\mathbb{R}$
$\hat{\mu}_1 + \sqrt{\frac{\ln(K/\delta)}{N}}$
$\hat{\mu}_1$
μ_1
$\hat{\mu}_1 - \sqrt{\frac{\ln(K/\delta)}{N}}$
$\hat{\mu}_2 + \sqrt{\frac{\ln(K/\delta)}{N}}$
$\hat{\mu}_2$
μ_2
$\hat{\mu}_2 - \sqrt{\frac{\ln(K/\delta)}{N}}$
$\hat{\mu}_3 + \sqrt{\frac{\ln(K/\delta)}{N}}$
μ_3
$\hat{\mu}_3$
$\hat{\mu}_3 - \sqrt{\frac{\ln(K/\delta)}{N}}$

Figure 1.2: Hoeffding inequalities for multiple arms.

Figure 1.2 qualitatively shows an example of how the Explore & Commit algorithm would fail: in this case we would choose the arm a_2 as the sample mean $\hat{\mu}_2$ is higher than the sample mean of the other arms, but we can clearly see how the true mean μ_3 of arm a_3 is higher than the true mean μ_2 of the arm a_2. If we choose the arm a_2, the loss of accumulated reward due to this suboptimal decision is called **regret** and is equal to:

$$\text{Regret}_{exploit} = (T - NK)(\mu_3 - \mu_2) \tag{1.8}$$

where $T - NK$ is the number of remaining steps dedicated to exploitation (total steps T minus those used to explore NK). The difference $(\mu_3 - \mu_2)$ is small, less than the confidence ranges.

Since we assumed that the reward range is $[0, 1]$, the maximum loss for a single round is at most 1, so the regret accumulated in the exploration phase is bounded above by:

$$\text{Regret}_{explore} \leq N(\underbrace{K-1}_{\text{one is optimal}}) \leq NK. \tag{1.9}$$

Let $\hat{I}$ be the empirical best arm and I^* the true best arm, the worst possible regret in exploitation is

$$\text{Regret}_{exploit} \leq (T - NK)(\mu_{I^*} - \mu_{\hat{I}}) \tag{1.10}$$

where

$$\mu_{I^*} - \mu_{\hat{I}} \leq \left[\hat{\mu}_{I^*} + \sqrt{\frac{\ln(K/\delta)}{N}}\right] - \left[\hat{\mu}_{\hat{I}} - \sqrt{\frac{\ln(K/\delta)}{N}}\right] \tag{1.11}$$

$$= \underbrace{\hat{\mu}_{I^*} - \hat{\mu}_{\hat{I}}}_{\leq 0} + 2\sqrt{\frac{\ln(K/\delta)}{N}} \leq 2\sqrt{\frac{\ln(K/\delta)}{N}} \tag{1.12}$$

hence the regret is at most twice our confidence interval:

$$\text{Regret}_{exploit} \leq 2T\sqrt{\frac{\ln(K/\delta)}{N}}. \tag{1.13}$$

The total regret is

$$\text{Regret}_T = \text{Regret}_{explore} + \text{Regret}_{exploit} \leq NK + 2T\sqrt{\frac{\ln(K/\delta)}{N}}. \tag{1.14}$$

We want to minimize the regret by choosing the optimal value of N, this can be done by taking the derivative with respect to N of the regret and setting it to zero, then we solve for N. This leads to:

$$N = \left(\frac{T\sqrt{\ln(K/\delta)}}{K}\right)^{\frac{2}{3}} \tag{1.15}$$

The following inequality holds

$$\text{Regret}_T = O(T^{\frac{2}{3}} K^{\frac{1}{3}} \ln^{\frac{1}{3}}(K/\delta)). \tag{1.16}$$

This **growth rate** of the regret using the Explore & Commit algorithm is kind of slow. We would like to get something faster, like $O(\sqrt{T})$ (that is proven to be the minimum we can get). We want to design a new algorithm, first of all we need a list of generic statistics useful to maintain in order to compute our confidence bounds and the regret:

- we denote $N_t(i)$ the number of times we have tried the arm a_i so far:

$$N_t(i) = \sum_{\tau=0}^{t-1} \mathbb{1}\{I_\tau = a_i\} \tag{1.17}$$

- the empirical mean so far:

$$\hat{\mu}_t(i) = \sum_{\tau=0}^{t-1} \mathbb{1}\{I_\tau = a_i\} \frac{r_\tau}{N_t(i)} \tag{1.18}$$

- the confidence with probability $1 - \delta$:

$$|\hat{\mu}_t(i) - \mu_i| \leq \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}. \tag{1.19}$$

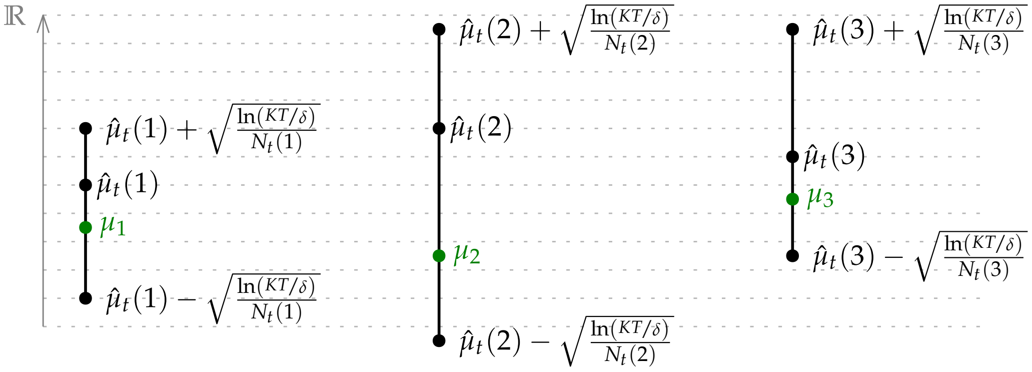

$\mathbb{R}$
$\hat{\mu}_t(1)+\sqrt{\frac{\ln(KT/\delta)}{N_t(1)}}$
$\hat{\mu}_t(1)$
μ_1
$\hat{\mu}_t(1)-\sqrt{\frac{\ln(KT/\delta)}{N_t(1)}}$
$\hat{\mu}_t(2)+\sqrt{\frac{\ln(KT/\delta)}{N_t(2)}}$
$\hat{\mu}_t(2)$
μ_2
$\hat{\mu}_t(2)-\sqrt{\frac{\ln(KT/\delta)}{N_t(2)}}$
$\hat{\mu}_t(3)+\sqrt{\frac{\ln(KT/\delta)}{N_t(3)}}$
$\hat{\mu}_t(3)$
μ_3
$\hat{\mu}_t(3)-\sqrt{\frac{\ln(KT/\delta)}{N_t(3)}}$

Figure 1.3: In the confidence interval in Equation (1.19), the length depends on how many times one arm has been tried.

For example, in Figure 1.3 the confidence interval of arm 2 is wider because we did not try arm 2 as many times as arm 1 and 3. We select the arm with the highest upper confidence bound, and we make an optimistic assumption: the true mean actually corresponds to the upper confidence bound.

- If the bound is large, it means either that arm has been tested few times, therefore if that is not good, the more we choose it the more the upper bound decreases and eventually another one will be the highest, or that it is actually good.
- If a confidence interval is large then we want to explore, a small interval on the selected arm means its high UCB reflects a genuinely high mean.

The **UCB algorithm** works as follows:

- For the first K iterations, we pull each arm once
- for $t = K, \dots, T-1$:
  - We pick the arm with the highest upper confidence bound:
$$I_t = \arg\max_{i=1,\dots,K} \left(\hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}} \right) \tag{1.20}$$
  - update the statistics.

Now let's compute the regret at a given time step t, we have:

$$\mu^* - \mu_{I_t} \le \hat{\mu}_t(I_t) + \sqrt{\frac{\ln(KT/\delta)}{N_t(I_t)}} - \mu_{I_t} \le 2\sqrt{\frac{\ln(KT/\delta)}{N_t(I_t)}} \tag{1.21}$$

if N_t is small, we have a high regret but we have to explore, if it's large, we exploit and the regret is small. The total regret summing over all iterations is:

$$\text{Regret}_T = \sum_{t=0}^{T-1} (\mu^* - \mu_{I_t}) \le \sum_{t=0}^{T-1} 2\sqrt{\frac{\ln(TK/\delta)}{N_t(I_t)}} \le 2\sqrt{\ln(TK/\delta)} \cdot \underbrace{\sum_{t=0}^{T-1} \sqrt{\frac{1}{N_t(I_t)}}}_{\le O(\sqrt{KT})} . \tag{1.22}$$

Theorem 1.1.1 : Regret of the UCB

With the UCB algorithm, the following bound holds:

$$\text{Regret}_T = O\left(\sqrt{KT \ln\left(\frac{KT}{\delta}\right)} \right) \tag{1.23}$$

or

$$\text{Regret}_T = \tilde{O}(\sqrt{KT}) \tag{1.24}$$

with probability $1-\delta$.

Proof: We want to bound the summation term appearing in the UCB regret bound. First we cluster the pulled arms into K groups, where the i-th group contains all steps where arm i is pulled:

$$\sum_{t=0}^{T-1}\sqrt{\frac{1}{n_t(I_t)}}=\sum_{i=1}^{K}\sum_{t=0}^{T-1}\mathbb{1}\{I_t=a_i\}\sqrt{\frac{1}{n_t(i)}} \tag{1.25}$$

where $n_t(i)$ is the size of the specific group.

$$\sum_{i=1}^{K}\sum_{t=0}^{T-1}\mathbb{1}\{I_t=a_i\}\sqrt{\frac{1}{n_t(i)}}=\sum_{i=1}^{K}\sum_{t=0}^{n_T(i)}\sqrt{\frac{1}{t}}. \tag{1.26}$$

We use the following inequality:

$$\sum_{k=1}^{N}\frac{1}{\sqrt{k}}\leq 2\sqrt{N} \tag{1.27}$$

hence from Equation (1.26) we have:

$$\sum_{i=1}^{K}\sum_{t=0}^{n_T(i)}\sqrt{\frac{1}{t}}\leq 2\sum_{i=1}^{K}\sqrt{n_T(i)}. \tag{1.28}$$

Since all $n_T(i)$ sum to T, by the Jensen's inequality (concavity of $\sqrt{\cdot}$) we have:

$$2\sum_{i=1}^{K}\sqrt{n_T(i)}=2K\left(\frac{1}{K}\sum_{i=1}^{K}\sqrt{n_T(i)}\right)\leq 2K\sqrt{\frac{1}{K}\sum_{i=1}^{K}n_T(i)}=2K\sqrt{\frac{T}{K}}=2\sqrt{KT}. \tag{1.29}$$

■

1.2 Contextual and Bayesian Bandits

We introduce the *context*: there is a set X of contexts/states, $x_t \in X$ is the state at time t, the reward depends not only on the action but also on the current state: $r(x_t, a_t)$. For simplicity we assume that the reward function is deterministic.

- At each time step, we are in the state x_t.
- We choose an action a_t and observe $r(x_t, a_t)$.
- The next state x_{t+1} does not depend on the action, but is sampled from a distribution μ.

A **policy** π is a mapping from *all* the states to the actions. Is important to emphasize that a policy must assign an action to **each possible state**; if it is not defined on a particular state, then it is not a policy.

- Can be deterministic: $a = \pi(s)$,
- or stochastic $\pi(a \mid s)$.

Note how we denoted s a state, this is a general notation. In the bandits context, we will use x to denote the state. We define the **optimal policy** as the one that maximizes the expected value for the reward:

$$\pi^\star=\arg\max_{\pi}\mathop{\mathbb{E}}_{x\sim\mu}r(x,\pi(x)). \tag{1.30}$$

If the set of states X is discrete the expected value is

$$\pi^\star=\arg\max_{\pi}\sum_{x\in X}r(x,\pi(x))\mu(x) \tag{1.31}$$

with $\mu(x)$[1] being the probability of sampling the state x.

---

[1] Formally, we should write $\mu(\{x\})$, since μ is a measure on X.

Let $\{\pi^0, \pi^1, \dots, \pi^{T-1}\}$ be the sequence of policies that we learned at each time step from the beginning to $t = T - 1$. At the T-th iteration, we define the **regret** as the total expected reward obtained using the optimal policy $\pi^\star$ minus the total expected reward obtained by using our sequence of policies:

$$\text{Regret}_T = T \mathop{\mathbb{E}}_{x \sim \mu} [r(x, \pi^\star(x))] - \sum_{t=0}^{T-1} \mathop{\mathbb{E}}_{x \sim \mu} [r(x, \pi^t(x))] \tag{1.32}$$

we recall that at every time step, the states are sampled from the distribution μ independently from the previous states.

1.2.1 Explore and Commit

Let's see how the explore & commit algorithm works on the contextual bandit, the goal is to build a *table* that assigns to each pair of state and action, the estimated expected value of the reward.

Algorithm 1 Explore & Commit

```
for $t = 0, \dots, N-1$ (explore) do
    observe the current state $x_t \sim \mu$
    sample a random action from a uniform distribution $a_t \sim \text{Unif}(A)$
    observe the reward $r_t = r(x_t, a_t)$
    build an unbiased estimator $\mathbf{r}_t$ defined as follows:
        $\mathbf{r}_t[a] = \begin{cases} 0 & \text{if } a \neq a_t \\ r_t \cdot |A| & \text{else} \end{cases}$    (1.33)
    at each time step $t$ the estimator assigns zero to all the entries of $\mathbf{r}_t$ except for the one used at time $t$, to which it assigns the reward multiplied by the number of actions.
end for
compute the policy as follows:
    $\hat{\pi} = \arg\max_{\pi} \sum_{i=0}^{N-1} \mathbf{r}_i[\pi(x_i)]$    (1.34)
for $t = N, \dots, T-1$ (commit) do
    observe the current state $x_t \sim \mu$
    play the arm $a_t = \hat{\pi}(x_t)$
end for
```

About the estimator: We want to give a description of how the estimator $\mathbf{r}_t$ works. There are N vectors $\mathbf{r}_0, \mathbf{r}_1, \mathbf{r}_2 \dots, \mathbf{r}_{N-1}$, each of dimension $|A|$ (the number of possible actions). For a given time step t, $\mathbf{r}_t$ is null on each action, except for the action chosen at time t:

$$\mathbf{r}_0 = \begin{pmatrix} r_0/p(a_0) \\ 0 \\ \vdots \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \mathbf{r}_1 = \begin{pmatrix} 0 \\ r_1/p(a_1) \\ \vdots \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \dots \quad, \mathbf{r}_t = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ r_t/p(a_t) \\ \vdots \\ 0 \end{pmatrix}, \quad \dots \quad, \mathbf{r}_{N-1} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \vdots \\ r_{N-1}/p(a_{N-1}) \end{pmatrix}. \tag{1.35}$$

$p(a_t)$ is the probability of choosing the action a_t, since they are sampled from a uniform distribution, for each action a we have $p(a) = \frac{1}{|A|}$ hence $\frac{r_t}{p(a_t)} = r_t \cdot |A|$. The regret for this algorithm has the following asymptotic behavior:

$$O(T^{2/3} \cdot |A|^{1/3} \cdot \ln^{1/3}(|\Pi|)) \tag{1.36}$$

where Π is the set of all the policies.

1.2.2 ϵ-Greedy

The ϵ-greedy algorithm never stops exploring, at each time step, with a given probability it selects a random action, else it selects the best action, exploiting the known information. We denote π^t the current policy that we are selecting as our best policy at time t. Let $\epsilon \in [0,1]$, if $\epsilon = 0$, we just exploit the available information, if $\epsilon = 1$, we explore by randomly choosing the actions.

Algorithm 2 ϵ-Greedy

```
for $t = 0, \dots, T-1$ do
    observe the current state $x_t \sim \mu$
    select a random number $c \sim \mathrm{Unif}([0,1])$
    if $c \leq \epsilon$ then
        sample a random action from a uniform distribution $a_t \sim \mathrm{Unif}(A)$ (explore)
    else
        choose the best action $a_t = \pi^t(x_t)$ (exploit)
    end if
    observe the reward $r_t = r(x_t, a_t)$
    build an unbiased estimator $\mathbf{r}_t$ defined as follows:
        $\mathbf{r}_t[a] = \begin{cases} 0 & \text{if } a \neq a_t \\ \frac{r_t}{p_t(a_t)} & \text{else} \end{cases}$    (1.37)
    where
        $p_t(a) = \frac{\epsilon}{|A|} + (1-\epsilon)\mathbb{1}\{a = \pi^t(x_t)\}$    (1.38)
    update the policy
        $\pi^{t+1} = \arg\max_{\pi} \sum_{i=0}^{t} \mathbf{r}_i[\pi(x_i)]$    (1.39)
end for
```

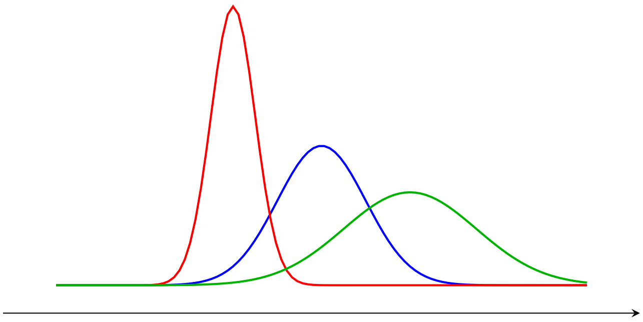
ϵ can also be time-varying, by making it high at the beginning (first explore) and then decaying as the time goes on (then exploit), for example $\epsilon(t) = 0.9\exp(-t) + 0.1$.

1.2.3 Gaussian Bayesian Bandits

In the previous discussion, we treated the reward r as deterministic. Now let r be random again. We denote $\nu(i)$ the reward distribution, for the specific action a_i (hence we have a reward distribution for each action), we are making a step back by removing the context, the following discussion assumes that there is only one state. We exploit prior knowledge by *assuming* that the distributions $\nu(i)$ are **Gaussians**:

$$\nu(i) = \mathcal{N}(\mu(i), \sigma^2(i)) \tag{1.40}$$

with $\mu(i)$ and $\sigma^2(i)$ being the mean and variance for the reward by choosing the action a_i.



Usually, at the beginning we assume as prior knowledge that the mean of all the Gaussians is zero and the standard deviation is 1. In the algorithm we will start exploring and trying different actions, sampling the rewards in order to fit the Gaussians with the correct values for the mean and the standard deviation.

Assume that we have fitted the Gaussians, we can apply the UCB to this case, the best action is the one with the highest upper bound for the reward, hence we choose the action with the Gaussian with the highest standard deviation (considering also the mean). Let K be the number of actions:

$$a_t = \underset{1 \leq i \leq K}{\arg\max}\, \mu_t(i) + \frac{c\sigma_t(i)}{\sqrt{N_t(i)}} \tag{1.41}$$

where

- $\mu_t(i), \sigma_t(i)$ are the current estimation at time t for the mean and the standard deviation of the distributions of the i-th action.
- c is an exploration parameter.
- $N_t(i)$ is the total number of times that the i-th action has been selected.

CHAPTER

2

PLANNING IN MDPS

2.1 Markov Decision Processes

First of all we introduce a *sequential* decision making approach. We see the agent that has to learn the policy as an entity that can interact with the world/environment through actions a_t, and can receive from the environment an observation o_t and a reward r_t after each action. The agent interacts at discrete time steps; such discrete interaction generates a **trajectory** (or history) of tuples (observation-action-reward):

$$\underbrace{h_t}_{\text{history at time } t} = (o_0, a_0, r_1, o_1, a_1, r_2, \dots, r_t, o_t, a_t). \tag{2.1}$$

We define a state function $s_t = f(h_t)$ that describes the current state of the agent at time t and depends on the history h_t and is typically hidden or unknown.

Definition 2.1.1 : Markov Assumption

A state s_t is **Markovian** if the future is independent of the past information, given the present state: the probability of ending up in a given state s_{t+1} by applying a_t at time t, depends only on the current state s_t and not on the history h_t:

$$\mathbb{P}(s_{t+1} \mid s_t, a_t) = \mathbb{P}(s_{t+1} \mid h_t, a_t). \tag{2.2}$$

A state can always be made Markovian if we define the state exactly as the previous history:

$$s_t = h_t. \tag{2.3}$$

In practice the current state is defined as the *latest observation*:

$$s_t = o_t \tag{2.4}$$

in this case the state is said to be **fully observable**.

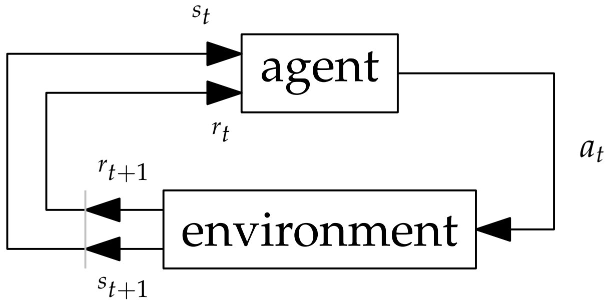


Figure 2.1: Markov Decision Process scheme.

A **Markov Decision Process (MDP)** is a sequential decision making scheme under the Markov assumption. The transition dynamics is generally stochastic: $\mathbb{P}(s_{t+1} \mid s_t, a_t)$. An alternative notation to denote the fact that the next state is stochastic by applying a_t on s_t is the following:

$$s_{t+1} \sim \mathbb{P}(\cdot \mid s_t, a_t) \tag{2.5}$$

$$\text{or} \quad s' \sim \mathbb{P}(\cdot \mid s, a) \tag{2.6}$$

A **reward** r_t is a scalar signal representing feedback on the actions taken by the agent, and indicates how well an agent is doing at a given time step t. The reward is a function of the state and action (reward taken by applying a given action at a given state):

$$r_{t+1} = R(s_t, a_t). \tag{2.7}$$

We define the **cost** as the opposite of the reward. Although in reinforcement learning the objective is always to maximize the reward function, the fact that this is the best way to reach a goal is not guaranteed, in fact the so-called **Reward Hypothesis**:

> *"all goals can be described as the maximization of expected cumulative reward."*

remains an open question today.

Definition 2.1.2 : Markov Decision Process

An MDP is defined by a 5-tuple:

$$(S, A, \mathbb{P}, R, \gamma) \tag{2.8}$$

where:

- S is the set of all possible states.
- A is the set of all the actions the agent can take.
- $\mathbb{P}$ is the transition probability, which defines the dynamics of the environment.
- R is the reward function.
- γ is the **discount factor**, a real value within the interval $[0, 1)$ that determines the importance of future rewards compared to immediate ones.

The behavior of an agent is given by a policy, a function that determines how agents select actions.

Definition 2.1.3 : Policy

A **policy** is a function $\pi : S \to \Delta(A)$ that assigns a distribution over the set of actions to every state.

A policy can be deterministic (special case where $\pi : S \to A$) and we write:

$$\pi(s) = a \tag{2.9}$$

or stochastic:

$$\pi(a \mid s) \in [0,1] \quad \text{with} \quad \sum_{a \in A} \pi(a \mid s) = 1. \tag{2.10}$$

We can also write $a \sim \pi(\cdot \mid s)$. Let's define the **probability of a trajectory**: What's the probability of seeing a given trajectory at time t according to π starting at s_0? We have the trajectory

$$(s_0, a_0, s_1, \ldots, s_t, a_t) \tag{2.11}$$

and

$$\begin{aligned} &\mathbb{P}^{\pi}(a_0, s_1, \ldots, s_t, a_t \mid s_0) && (2.12)\\ =&\pi(a_0 \mid s_0)\mathbb{P}(s_1 \mid s_0, a_0)\pi(a_1 \mid s_1)\mathbb{P}(s_2 \mid s_1, a_1)\ldots\mathbb{P}(s_t \mid s_{t-1}, a_{t-1})\pi(a_t \mid s_t) && (2.13)\end{aligned}$$

notice how we added π as superscript to denote the fact that this probability depends on the given policy function. We can also define the **state visitation probability**, i.e. the probability of visiting a particular state $\hat{s}$ at time t according to π starting from s_0:

$$\mathbb{P}_t^{\pi}(s_t = \hat{s} \mid s_0) = \sum_{a_0, s_1 \ldots a_{t-1}} \mathbb{P}^{\pi}(s_0, a_0, s_1, \ldots, s_t = \hat{s}). \tag{2.14}$$

The summation in equation (2.14) sums over all trajectories of length t that start at s_0 and end in $\hat{s}$.

A basic example of a Markov decision process is a robotic manipulator with a hand gripper that can grab and carry a ball:

- the states are the possible robot configurations (joint values) and the ball position,
- the actions are the possible torques on the arm and the finger joints,
- the transition is stochastic, given by the physics (how the torques update the joint values over time) and by the noise,
- the policy maps each configuration+ball position to a torque for the joints,
- the cost (opposite of the reward) is the magnitude of the torque and the distance to the goal.

2.1.1 MDP Horizon

In Definition 2.1.2 we introduced a discount factor $\gamma \in [0,1)$. This term weights the importance of immediate rewards with respect to future rewards:

- $\gamma = 0$: Our agent only cares about immediate rewards.
- $\gamma = 1$: Immediate and future rewards are equally important.

The discount factor is used in the **Value Function** V^{π}, a function that estimates the goodness of a state based on its value. It is also a measure to compare different policies.

Definition 2.1.4 : Value Function

The **Value Function** V^{π} measures the expected cumulative reward (the sum of all future rewards, possibly discounted by the γ factor) starting from s and continuing to follow the policy π for all subsequent actions.

$$\begin{aligned} V^{\pi}(s) =&\mathbb{E}_{\pi}\left[\sum_{h=0}^{\infty} \gamma^h r_{h+1} \mid s_0 = s\right] && (2.15)\\ =&\mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h \cdot \underbrace{r_{h+1}}_{R(s_h,a_h)} \mid s_0 = s,\ a_h \sim \pi(\,\cdot \mid s_h),\ s_{h+1} \sim \mathbb{P}(\cdot \mid s_h, a_h)\right]. && (2.16)\end{aligned}$$

With $\mathbb{E}$ we denote the expected value. The value function V^π considers all the possible infinite trajectories that are generated by applying π starting from the argument s, since the transition and the policy are (or might be) stochastic, a probability is assigned to each trajectory, so the expected value is considered, i.e. the sum over the cumulative reward of a given trajectory, weighted by the probability of that trajectory.

Example: Let's assume that there is an MDP with three possible trajectories starting from s_a:

$$\text{trajectory } 1 : (s_a, a, s_c) \tag{2.17}$$
$$\text{trajectory } 2 : (s_a, a, s_b, a, s_c) \tag{2.18}$$
$$\text{trajectory } 3 : (s_a, a, s_b, a, s_b, a, s_c) \tag{2.19}$$

There is only one action (always chosen by the policy) and the reward function is deterministic:

$$R(s_a, a) = 2 \tag{2.20}$$
$$R(s_b, a) = 1 \tag{2.21}$$
$$R(s_c, a) = 0 \tag{2.22}$$

Usually these trajectories are infinite, but in this case we assume that they all end up in s_c, an ending state where all actions have a reward of zero with probability 1:

$$\mathbb{P}(s_c | s_c, a) = 1. \tag{2.23}$$

The expected value of the cumulative reward assuming the following probabilities:

$$\mathbb{P}(s_a, a, s_c) = 0.2 \tag{2.24}$$
$$\mathbb{P}(s_a, a, s_b, a, s_c) = 0.1 \tag{2.25}$$
$$\mathbb{P}(s_a, a, s_b, a, s_b, a, s_c) = 0.7 \tag{2.26}$$

assuming $\gamma = 1$ is

$$2 \cdot 0.2 + 3 \cdot 0.1 + 4 \cdot 0.7 = 3.5. \tag{2.27}$$

The Action-Value function Q^π goes one step further than V^π, tying the value not only to the state, but also to the action you choose to take.

Definition 2.1.5 : Action-Value Function

The **Action-Value Function** Q^π measures the expected value of the cumulative reward starting from a state s, performing a specific action a and then continuing to follow the policy π.

$$Q^\pi(s, a) = \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h \cdot \underbrace{r_{h+1}}_{R(s_h, a_h)} \mid (s_0, a_0) = (s, a),\ \underbrace{a_h \sim \pi(\,\cdot \mid s_h)}_{\forall\, h \geq 1},\ s_{h+1} \sim \mathbb{P}(\cdot \mid s_h, a_h)\right]. \tag{2.28}$$

Theorem 2.1.1 : The Bellman Equation

The value of a certain state can be expanded in terms of the current reward and the value of the next states according to the policy:

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V^\pi(s')\right]. \tag{2.29}$$

The Bellman equation can be applied to the action-value function:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V^\pi(s'). \tag{2.30}$$

In Equation (2.29) V^π is a function of s and $\pi(s)$ while in equation (2.30) Q^π is a function of s and a (as highlighted by the green characters). As a result of these equations we have the following identity:

$$V^\pi(s) = \sum_{a\in A} \pi(a \mid s) Q^\pi(s,a) \tag{2.31}$$

if π is deterministic:

$$V^\pi(s) = Q^\pi(s, \pi(s)) \tag{2.32}$$

V^π can be expressed in terms of Q^π. One last quantity that we introduce is the following.

Definition 2.1.6 : Discounted State-Action Distribution

The discounted state-action distribution quantifies how often (and with what discounted time weight) the pair (s,a) is visited along the entire infinite horizon.

$$d^\pi_{s_0}(s,a) = (1-\gamma)\sum_{h=0}^{\infty} \gamma^h \mathbb{P}^\pi(s_h = s \mid s_0)\pi(a \mid s). \tag{2.33}$$

Since there is a discount factor $\gamma < 1$, events that occur far away in time matter less and less.

$\mathbb{P}^\pi(s_h = s \mid s_0)\pi(a \mid s)$ is the exact probability of being in state s and executing action a exactly at time step h, conditioned on having started in s_0 and having followed the π policy for all previous steps.

2.2 Optimality of a Policy

For an infinite horizon MDP there always exists a deterministic **optimal policy** $\pi^\star$ that satisfies:

$$V^{\pi^\star}(s) \geq V^\pi(s), \quad \forall s,\ \forall \pi \tag{2.34}$$

This means that the policy $\pi^\star$ is the one that guarantees the greatest expected reward, since it dominates all other policies in each state. We will use also the following notation:

$$V^{\pi^\star} = V^* \tag{2.35}$$

$$Q^{\pi^\star} = Q^*. \tag{2.36}$$

The **Bellman Optimality** condition states that:

$$V^*(s) = \max_{a\in A} Q^*(s,a). \tag{2.37}$$

Example: Consider the MDP in Figure 2.2.

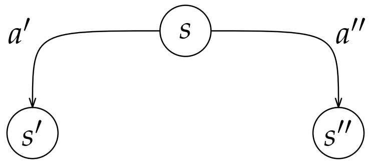


Figure 2.2: Three-states MDP example.

We assume that V^* is known in the states s' and s''. We want to try a' and a'' in s in order to get the rewards and compute Q^* in s for the actions:

- We apply a' and get $R(s,a')$. We can compute $Q^*(s,a') = R(s,a') + \gamma \underbrace{V^*(s')}_{\text{known}}$.
- We apply a'' and get $R(s,a'')$. We can compute $Q^*(s,a'') = R(s,a'') + \gamma \underbrace{V^*(s'')}_{\text{known}}$.

Now we can use the identity in Equation (2.37) to compute $V^*(s)$:

$$V^*(s) = \max\{Q^*(s,a'),\ Q^*(s,a'')\}. \tag{2.38}$$

Theorem 2.2.1 : Bellman Optimality (Existence)

Let $\gamma \in [0,1)$. The optimal value function $V^*(s)$ satisfies the nonlinear Bellman equation:

$$V^*(s) = \max_{a\in A}\left[R(s,a) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,a)V^*(s')\right] \tag{2.39}$$

hence the policy:

$$\hat{\pi}(s) = \arg\max_{a\in A} Q^*(s,a) \tag{2.40}$$

is the optimal policy: $V^{\hat{\pi}} = V^*$.

Proof: First we prove that V^* satisfies the Bellman Equation (2.39). For a generic policy π, we can write the Bellman equation by isolating the first action $a \sim \pi(\cdot \mid s)$:

$$V^\pi(s) = \sum_{a\in A} \pi(a \mid s)\left[R(s,a) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,a)V^\pi(s')\right]. \tag{2.41}$$

The optimal value function can be written as follows:

$$V^*(s) = \max_{\pi} \sum_{a\in A} \pi(a \mid s)\left[R(s,a) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,a)V^\pi(s')\right]. \tag{2.42}$$

Consequently, the maximum over π coincides with the maximization over a single action $a \in A$ by placing $V^*(s')$ in the future steps:

$$V^*(s) = \max_{a\in A}\left[R(s,a) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,a)V^*(s')\right] \tag{2.43}$$

hence V^* satisfies the Bellman equation. Now we have to prove that $\hat{\pi}$ is the optimal policy. We define the deterministic policy $\hat{\pi}$ as in Equation (2.40). Since $\hat{\pi}(s)$ selects the action that maximizes Q^* we can write:

$$\begin{aligned} V^*(s) =& \max_{a\in A}\left[R(s,a) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,a)V^*(s')\right] && (2.44)\\ =& R(s,\hat{\pi}(s)) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))V^*(s'). && (2.45) \end{aligned}$$

On the other hand, from the Bellman expectation equation applied to $\hat{\pi}$ we can write:

$$V^{\hat{\pi}}(s) = R(s,\hat{\pi}(s)) + \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))V^{\hat{\pi}}(s') \tag{2.46}$$

if we subtract (2.46) from (2.45) we get:

$$V^*(s) - V^{\hat{\pi}}(s) = \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))\Big(V^*(s') - V^{\hat{\pi}}(s')\Big). \tag{2.47}$$

Now we take the absolute value of both sides:

$$|V^*(s) - V^{\hat{\pi}}(s)| = \Big|\gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))\Big(V^*(s') - V^{\hat{\pi}}(s')\Big)\Big|. \tag{2.48}$$

By the triangle inequality:

$$\begin{aligned} |V^*(s) - V^{\hat{\pi}}(s)| =& \Big|\gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))\Big(V^*(s') - V^{\hat{\pi}}(s')\Big)\Big| && (2.49)\\ \le& \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))\big|V^*(s') - V^{\hat{\pi}}(s')\big| && (2.50)\\ \le& \gamma \sum_{s'\in S} \mathbb{P}(s' \mid s,\hat{\pi}(s))\|V^* - V^{\hat{\pi}}\|_\infty && (2.51) \end{aligned}$$

we used the L_∞ norm, the following inequality holds:

$$|V^*(s) - V^{\hat{\pi}}(s)| \leq \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, \hat{\pi}(s)) \|V^* - V^{\hat{\pi}}\|_\infty. \tag{2.52}$$

$\|V^* - V^{\hat{\pi}}\|_\infty$ is constant and $\sum_{s' \in S} \mathbb{P}(s' \mid s, \hat{\pi}(s)) = 1$, hence

$$|V^*(s) - V^{\hat{\pi}}(s)| \leq \gamma \|V^* - V^{\hat{\pi}}\|_\infty. \tag{2.53}$$

this holds for each s hence:

$$\|V^* - V^{\hat{\pi}}\|_\infty \leq \gamma \|V^* - V^{\hat{\pi}}\|_\infty \implies \tag{2.54}$$

$$\|V^* - V^{\hat{\pi}}\|_\infty = 0 \implies \tag{2.55}$$

$$V^{\hat{\pi}}(s) = V^*(s), \quad \forall s. \tag{2.56}$$

■

Theorem 2.2.2 : Bellman Optimality (Uniqueness)

Let $\gamma \in [0, 1)$. For any value function V, if

$$V(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V(s') \right] \quad \forall s \in S \tag{2.57}$$

then $V(s) = V^*(s)$ for all $s \in S$.

Proof: Let V be any value function satisfying the Bellman optimality equation for all $s \in S$. Since V^* is also a solution, we can write (for any state s):

$$|V(s) - V^*(s)| \tag{2.58}$$

$$= \left| \max_{a \in A} \left(R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V(s') \right) - \max_{a \in A} \left(R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V^*(s') \right) \right|. \tag{2.59}$$

Using the property $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$:

$$|V(s) - V^*(s)| \tag{2.60}$$

$$\leq \max_{a \in A} \left| \left(R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V(s') \right) - \left(R(s, a) + \gamma \sum_{s' \in S} \mathbb{P}(s' \mid s, a) V^*(s') \right) \right| \tag{2.61}$$

we can cancel out $R(s, a)$ and move γ outside the max function:

$$|V(s) - V^*(s)| \leq \gamma \max_{a \in A} \left| \sum_{s' \in S} \mathbb{P}(s' \mid s, a) \big(V(s') - V^*(s') \big) \right| \tag{2.62}$$

by the triangle inequality we can move the absolute value inside the sum:

$$|V(s) - V^*(s)| \leq \gamma \max_{a \in A} \sum_{s' \in S} \mathbb{P}(s' \mid s, a) \left| V(s') - V^*(s') \right|. \tag{2.63}$$

We use the definition of L_∞ norm:

$$|V(s') - V^*(s')| \leq \max_{y \in S} |V(y) - V^*(y)| = \|V - V^*\|_\infty \tag{2.64}$$

Equation (2.63) becomes:

$$|V(s) - V^*(s)| \leq \gamma \max_{a \in A} \sum_{s' \in S} \mathbb{P}(s' \mid s, a) \|V - V^*\|_\infty \tag{2.65}$$

$\|V - V^*\|_\infty$ is a constant hence can be moved out of the summation:

$$|V(s) - V^*(s)| \leq \gamma \max_{a \in A} \left(\|V - V^*\|_\infty \sum_{s' \in S} \mathbb{P}(s' \mid s, a) \right) \tag{2.66}$$

trivially

$$\sum_{s' \in S} \mathbb{P}(s' \mid s, a) = 1$$

hence

$$|V(s) - V^*(s)| \leq \gamma \max_{a \in A} \left(\|V - V^*\|_\infty \cdot 1 \right) = \gamma \|V - V^*\|_\infty \tag{2.67}$$

since this inequality holds for each state $s \in S$, we can write:

$$\underbrace{\max_{s \in S} |V(s) - V^*(s)|}_{\|V-V^*\|_\infty} \leq \gamma \|V - V^*\|_\infty \implies \tag{2.68}$$

$$\|V - V^*\|_\infty \leq \gamma \|V - V^*\|_\infty \implies \tag{2.69}$$

$$(1-\gamma)\|V - V^*\|_\infty \leq 0 \implies \|V - V^*\|_\infty = 0. \tag{2.70}$$

This holds since $\gamma \in [0, 1)$, hence $(1 - \gamma) > 0$. By definition:

$$\|V - V^*\|_\infty = 0 \implies \tag{2.71}$$

$$V(s) = V^*(s) \quad \forall s \in S. \tag{2.72}$$

■

This formulation reduces the multi-step problem to a one-step greedy choice plus the continuation value $V(s')$. By uniqueness of the fixed point, any function V satisfying this optimality equation coincides with V^*.

2.3 Value iteration

We have seen how the value function estimates the goodness of a state based on its value, by applying a given policy, hence it is also a measure to compare policies.

$$V^\pi(s_t) = \mathbb{E}_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} \cdots \mid s_t] = \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r_h \mid s_0 = s_t,\ a_h = \pi(s_h),\ s_{h+1} \sim p(\cdot \mid s_h, a_h)\right]. \tag{2.73}$$

About the notation: from now we will also use $p(\cdot \mid s_h, a_h)$ to denote the probability instead of the symbol $\mathbb{P}$.

2.3.1 Exact Policy Evaluation

In this section we will describe a method to *compute exactly the value function* V^π of a given policy π, by assuming that the reward function is deterministic, and the transition probabilities are known. For all the states, the Bellman expectation equation holds:

$$V^\pi(s) = r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,\pi(s))}[V^\pi(s')] \tag{2.74}$$

with the explicit expression of the expected value being:

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} V^\pi(s') p(s'|s, \pi(s)) \tag{2.75}$$

this can be written as a system of $|S|$ equations (one for each state), hence the value function V^π and the rewards $\mathbf{r}^\pi$ can be seen as vectors in $\mathbb{R}^m$:

$$\mathbf{V}^\pi = \begin{pmatrix} V^\pi(s_1) \\ \vdots \\ V^\pi(s_m) \end{pmatrix}, \quad \mathbf{r}^\pi = \begin{pmatrix} r(s_1, \pi(s_1)) \\ \vdots \\ r(s_m, \pi(s_m)) \end{pmatrix} \tag{2.76}$$

by defining $m = |S|$. Then, for the transition probabilities we have the $m \times m$ matrix P^π defined as follows:

$$P^\pi(i,j) = p(s_j \mid s_i, \pi(s_i)) \tag{2.77}$$

The element at row i and column j represents the conditional probability $p(s_j|s_i, \pi(s_i))$ of moving from state s_i to state s_j (given by the transition function):

$$P^\pi = \begin{pmatrix} p(s_1 \mid s_1, \pi(s_1)) & \dots & p(s_m \mid s_1, \pi(s_1)) \\ \vdots & \ddots & \vdots \\ p(s_1 \mid s_m, \pi(s_m)) & \dots & p(s_m \mid s_m, \pi(s_m)) \end{pmatrix} \tag{2.78}$$

hence the Bellman expectation can be expressed in matrix form:

$$\mathbf{V}^\pi = \mathbf{r}^\pi + \gamma P^\pi \mathbf{V}^\pi \tag{2.79}$$

if the rewards $\mathbf{r}^\pi$ and the transition probabilities P^π are known, we can solve this system of equations for $\mathbf{V}^\pi$ by inverting the matrix:

$$\mathbf{V}^\pi = (I - \gamma P^\pi)^{-1} \mathbf{r}^\pi. \tag{2.80}$$

If $m = |S|$ is big (large number of states) this inversion becomes computationally expensive, since the number of operations is in $O(m^3)$.

2.3.2 Fixed Point Iteration

We will describe a computationally less expensive method to compute $V^\pi(s)$ for each s. Given a metric space M and a function $f : M \to M$, a point $\mathbf{x} \in M$ is called **fixed** if:

$$f(\mathbf{x}) = \mathbf{x}. \tag{2.81}$$

These points are related to a particular class of functions.

Definition 2.3.1 : Contraction Mapping

A function $f : M \to M$ is a contraction mapping if there exists a constant $k \in [0, 1)$ such that, for every pair of points $\mathbf{x}, \mathbf{y}$ the following condition holds:

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq k \|\mathbf{x} - \mathbf{y}\| \tag{2.82}$$

The following result is crucial.

Theorem 2.3.1 : Banach

If $f : M \to M$ is a contraction mapping and M is complete, then there exists a unique fixed point $\mathbf{x}^* \in M$

$$f(\mathbf{x}^*) = \mathbf{x}^*$$

and, by starting from an arbitrary $\mathbf{x}_0 \in M$, and applying iteratively

$$\mathbf{x}_{i+1} = f(\mathbf{x}_i)$$

for $i = 0, 1, \dots \infty$, the sequence will converge to $\mathbf{x}^*$.

In the simplest case, the contraction mapping f is just a matrix O:

$$\|O\mathbf{x} - O\mathbf{y}\| \leq \gamma \|\mathbf{x} - \mathbf{y}\| \tag{2.83}$$

instead of k we are using the discount factor $\gamma \in [0, 1)$. We will use the fact that the value function $\mathbf{V}^\pi$ **is a fixed point** of a given operator, and can be found by applying that operator (that is a contraction mapping) iteratively.

The metric space we work in, of which $\mathbf{V}^\pi$ is part, is $\mathbb{R}^m$. The contraction mapping $\mathcal{B}$ that we will use is the following:

$$\mathcal{B}(\mathbf{x}) = \mathbf{r}^\pi + \gamma P^\pi \mathbf{x} \tag{2.84}$$

this is an affine operator, defined through the rewards $\mathbf{r}^\pi$ and the probabilities of the transitions P^π. The value function V^π is **the fixed point of** $\mathcal{B}$. Hence we have an iterative algorithm.

Algorithm 3 Fixed Point Iteration

```
initialize $\mathbf{V}_0 = \mathbf{0}$
let $t = 0$
do
    $\mathbf{V}_{t+1} = \mathbf{r}^\pi + \gamma P^\pi \mathbf{V}_t$
    $t \leftarrow t + 1$
while convergence is not met
```

We can stop the iterations when the difference between $\mathbf{V}_{t+1}$ and $\mathbf{V}_t$ is less than a defined threshold ϵ:

$$\text{stop if } \|\mathbf{V}_{t+1} - \mathbf{V}_t\| \leq \epsilon. \tag{2.85}$$

We have to prove the fact that the Algorithm 3 converges to the value function $\mathbf{V}^\pi$.

Theorem 2.3.2 : Iterative Policy Evaluation

Let $\mathbf{V}_0$ be our starting guess for the value function $\mathbf{V}^\pi$, and

$$\mathbf{V}_{t+1} = \mathcal{B}(\mathbf{V}_t). \tag{2.86}$$

We denote $V_t(s)$ the component of $\mathbf{V}_t$ relative to the state s. For each single state $s \in S$ we have:

$$|[\mathcal{B}V_t](s) - [BV^\pi](s)| = |V_{t+1}(s) - V^\pi(s)| \leq \gamma^{t+1} \|\mathbf{V}_0 - \mathbf{V}^\pi\|_\infty. \tag{2.87}$$

Proof: If Equation (2.87) holds, by applying $\mathcal{B}$ for a large number of steps $(t \to \infty)$, we would have:

$$\lim_{t\to\infty} |V_{t+1}(s) - V^\pi(s)| \leq \lim_{t\to\infty} \gamma^{t+1} \|\mathbf{V}_0 - \mathbf{V}^\pi\| \implies \lim_{t\to\infty} |V_{t+1}(s) - V^\pi(s)| = 0 \tag{2.88}$$

$$\implies \lim_{t\to\infty} V_t(s) = V^\pi(s), \quad \forall s \in S \tag{2.89}$$

Let's start with the proof, the following reasoning holds for all the states $s \in S$. We have

$$|V_{t+1}(s) - V^\pi(s)| \tag{2.90}$$

we expand $V_{t+1}(s)$ and $V^\pi(s)$:

$$V_{t+1}(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} V_t(s') p(s' \mid s, \pi(s)) \tag{2.91}$$

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} V^\pi(s') p(s' \mid s, \pi(s)) \tag{2.92}$$

hence we have

$$|V_{t+1}(s) - V^\pi(s)| \tag{2.93}$$

$$= \left| r(s, \pi(s)) + \gamma \sum_{s' \in S} V_t(s') p(s' \mid s, \pi(s)) - r(s, \pi(s)) - \gamma \sum_{s' \in S} V^\pi(s') p(s' \mid s, \pi(s)) \right| \tag{2.94}$$

$r(s, \pi(s))$ can be canceled:

$$\left|\gamma \sum_{s' \in S} V_t(s') p(s' \mid s, \pi(s)) - \gamma \sum_{s' \in S} V^\pi(s') p(s' \mid s, \pi(s))\right| \tag{2.95}$$

the two terms are summing over the same set hence:

$$\left|\gamma \sum_{s' \in S} V_t(s') p(s' \mid s, \pi(s)) - V^\pi(s') p(s' \mid s, \pi(s))\right| \tag{2.96}$$

we apply the triangle inequality:

$$\left|\gamma \sum_{s' \in S} V_t(s') p(s' \mid s, \pi(s)) - V^\pi(s') p(s' \mid s, \pi(s))\right| \tag{2.97}$$

$$\leq \gamma \sum_{s' \in S} \left|V_t(s') p(s' \mid s, \pi(s)) - V^\pi(s') p(s' \mid s, \pi(s))\right| \tag{2.98}$$

we can group $p(s' \mid s, \pi(s))$

$$\gamma \sum_{s' \in S} \left|p(s' \mid s, \pi(s)) \big(V_t(s') - V^\pi(s')\big)\right| \tag{2.99}$$

since

$$|V_t(s') - V^\pi(s')| \leq \|V_t - V^\pi\|_\infty$$

we can proceed as follows:

$$\gamma \sum_{s' \in S} \left|p(s' \mid s, \pi(s)) \big(V_t(s') - V^\pi(s')\big)\right| \tag{2.100}$$

$$\leq \gamma \sum_{s' \in S} p(s' \mid s, \pi(s)) \|V_t - V^\pi\|_\infty \tag{2.101}$$

$\|V_t - V^\pi\|_\infty$ is constant hence we can move it out of the sum and get:

$$\gamma \sum_{s' \in S} p(s' \mid s, \pi(s)) \|V_t - V^\pi\|_\infty \tag{2.102}$$

$$= \gamma \|V_t - V^\pi\|_\infty \underbrace{\sum_{s' \in S} p(s' \mid s, \pi(s))}_{\text{sum up to 1}} = \gamma \|V_t - V^\pi\|_\infty. \tag{2.103}$$

We have obtained the following inequality

$$|V_{t+1}(s) - V^\pi(s)| \leq \gamma \|V_t - V^\pi\|_\infty, \quad \forall s \in S \tag{2.104}$$

since it holds for each s, we switch to the L_∞ norm:

$$\|V_{t+1} - V^\pi\|_\infty \leq \gamma \|V_t - V^\pi\|_\infty \tag{2.105}$$

hence:

$$\|V_{t+1} - V^\pi\|_\infty \leq \gamma \|V_t - V^\pi\|_\infty \leq \gamma^{t+1} \|V_0 - V^\pi\|_\infty. \tag{2.106}$$

■

The computational cost of finding an ϵ-accurate value function is in $O(m^2 \ln \frac{1}{\epsilon})$.

Now that we know how to compute the value function of a given policy, we are interested in finding the optimal policy π^*. We can perform a brute force search, by listing all the possible policies and evaluating their value function, but this would be computationally unfeasible, in the order of $O(|A|^m \cdot m^3)$, where $m = |S|$. We can use the Bellman optimality equation:

$$V^*(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} p(s' \mid s, a) V^*(s')\right]. \tag{2.107}$$

We define the operator $\mathcal{B}$ as the Bellman optimality equation:

$$[\mathcal{B}V](s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} p(s' \mid s,a) V(s') \right]. \tag{2.108}$$

we call this operator the **Bellman optimality operator**, it acts on a “value function” V.

Observation 2.3.1

Note that for a fixed policy, there is a unique value function corresponding to it, rather than multiple ones. As we stated, the Bellman optimality operator acts on a “value function”, meaning it maps a function that assigns a real number to each state (which can also be viewed as a vector in $\mathbb{R}^m$) to another function of the same type. Specifically, the Bellman optimality operator takes a value function V and produces a new value function $\mathcal{B}V$. Hence, it can be seen as a mapping from $\mathbb{R}^m$ to $\mathbb{R}^m$, though in our context it is more accurate to think of it as an operator acting on the space of “possible” value functions.

We recall that the infinity norm of a value function is $\|V\|_\infty = \max_s |V(s)|$.

Theorem 2.3.3 : Bellman Optimality Operator is a Contraction

Given two value functions V_k, V_j (that can be seen as vector in $\mathbb{R}^m$), then:

$$\|\mathcal{B}V_k - \mathcal{B}V_j\|_\infty \leq \gamma \|V_k - V_j\|_\infty \tag{2.109}$$

Proof:

$$\|\mathcal{B}V_k - \mathcal{B}V_j\|_\infty \tag{2.110}$$

$$= \left\| \max_a \left(r(s,a) + \gamma \sum_{s' \in S} p(s' \mid s,a) V_k(s') \right) - \max_{a'} \left(r(s,a') + \gamma \sum_{s' \in S} p(s' \mid s,a') V_j(s') \right) \right\|_\infty \tag{2.111}$$

$$\leq \max_a \left\| \left(r(s,a) + \gamma \sum_{s' \in S} p(s' \mid s,a) V_k(s') - r(s,a) - \gamma \sum_{s' \in S} p(s' \mid s,a) V_j(s') \right) \right\|_\infty \tag{2.112}$$

$$= \max_a \left\| \gamma \sum_{s' \in S} p(s' \mid s,a)(V_k(s') - V_j(s')) \right\|_\infty \tag{2.113}$$

$$\leq \max_a \left\| \gamma \sum_{s' \in S} p(s' \mid s,a) \|V_k - V_j\| \right\|_\infty \tag{2.114}$$

$$= \max_a \left\| \gamma \|V_k - V_j\| \sum_{s' \in S} p(s' \mid s,a) \right\|_\infty \tag{2.115}$$

$$= \gamma \|V_k - V_j\|_\infty \tag{2.116}$$

■

The optimality operator is a γ-contraction, it makes value functions closer by a factor of at least γ. Now we define the Bellman operator for the Q function. We recall that Q assigns a value to each pair state-action. The operator is defined as follows:

$$[\mathcal{B}Q](s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot \mid s,a)} \left(\max_{a'} [Q(s',a')] \right). \tag{2.117}$$

Since $Q : S \times A \to \mathbb{R}$, then also $\mathcal{B}Q : S \times A \to \mathbb{R}$.

Proposition 2.3.1

The Q-function associated to the optimal policy, denoted Q^*, is a fixed point of the Bellman operator $\mathcal{B}$, that is a contraction mapping.

This is a crucial result, we can obtain the Q-function of the optimal policy just by applying the operator iteratively.

Algorithm 4 Value Iteration

```
initialize $Q_0$ as
    $$Q_0(s,a) = 0, \quad \begin{matrix} \forall s \in S \\ \forall a \in A \end{matrix}$$
let $t = 0$
do
    $Q_{t+1} = \mathcal{B}Q_t$
    $t \leftarrow t + 1$
while convergence is not met
```

The correctness is guaranteed by the fact that the operator is a contraction mapping:

$$\|Q_{t+1} - Q^*\|_\infty = \|\mathcal{B}Q_t - Q^*\|_\infty \leq \gamma\|Q_t - Q^*\|_\infty \leq \gamma^{t+1}\|Q_0 - Q^*\|_\infty \tag{2.118}$$

Now that we know how to find the Q-function of the optimal policy, we have to define the optimal policy, this is easy. Assume that we have executed Algorithm 4 for i iterations, if i is sufficiently large we have

$$Q_i(s,a) \simeq Q^*(s,a), \quad \begin{matrix} \forall s \in S \\ \forall a \in A \end{matrix} \tag{2.119}$$

at this point we define the policy (relative to i iterations) as

$$\pi_i(s) = \arg\max_{a \in A} Q_i(s,a). \tag{2.120}$$

the quality of this policy is described by the following result.

Proposition 2.3.2 : Value Iteration

Let Q_i be the Q-function at the i-th iteration of Algorithm 4, and let π_i be its associated policy. The value function V^{π_i} for policy π_i is related to the optimal value function V^* as follows:

$$V^{\pi_i}(s) \geq V^*(s) - \frac{2\gamma^i}{1-\gamma}\|Q_0 - Q^*\|_\infty, \quad \forall s \in S \tag{2.121}$$

hence

$$\lim_{i \to \infty} V^{\pi_i} = V^* \implies \lim_{i \to \infty} \pi_i = \pi^*. \tag{2.122}$$

Proof: The following reasoning holds for each $s \in S$, we consider:

$$V^{\pi_i}(s) - V^*(s) \tag{2.123}$$

the value function can be rewritten in terms of the Q-function:

$$V^{\pi_i}(s) - V^*(s) = Q^{\pi_i}(s, \pi_i(s)) - Q^*(s, \pi^*(s)) \tag{2.124}$$

$$= Q^{\pi_i}(s, \pi_i(s)) \underbrace{- Q^*(s, \pi_i(s)) + Q^*(s, \pi_i(s))}_{\text{add and subtract}} - Q^*(s, \pi^*(s)) \tag{2.125}$$

now we take the first two terms $Q^{\pi_i}(s,\pi_i(s)) - Q^*(s,\pi_i(s))$ and we expand them with the Bellman equation:

$$\begin{aligned}
&\underbrace{Q^{\pi_i}(s,\pi_i(s)) - Q^*(s,\pi_i(s))}_{\text{expand}} + Q^*(s,\pi_i(s)) - Q^*(s,\pi^*(s)) && (2.126)\\
=&\gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] + Q^*(s,\pi_i(s)) - Q^*(s,\pi^*(s)) && (2.127)\\
& && (2.128)
\end{aligned}$$

now we add $-Q_i(s,\pi_i(s)) + Q_i(s,\pi^*(s))$:

$$\begin{aligned}
&\gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] + Q^*(s,\pi_i(s)) - Q^*(s,\pi^*(s)) && (2.129)\\
\geq&\gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] + Q^*(s,\pi_i(s)) - Q_i(s,\pi_i(s)) + Q_i(s,\pi^*(s)) - Q^*(s,\pi^*(s)) && (2.130)
\end{aligned}$$

we switched to the inequality $\geq$ since $-Q_i(s,\pi_i(s)) + Q_i(s,\pi^*(s))$ is not positive:

$$Q_i(s,\pi_i(s)) \geq Q_i(s,\pi^*(s))$$

now we use the L_∞ norm:

$$\gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] + \underbrace{Q^*(s,\pi_i(s)) - Q_i(s,\pi_i(s))}_{\geq \|Q_0 - Q^*\|_\infty} + \underbrace{Q_i(s,\pi^*(s)) - Q^*(s,\pi^*(s))}_{\geq \|Q_0 - Q^*\|_\infty} \quad (2.131)$$

$$\geq \gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] - 2\|Q_i - Q^*\|_\infty \quad (2.132)$$

by exploiting $\|Q_i - Q^*\|_\infty \leq \gamma^i \|Q_0 - Q^*\|_\infty$:

$$\begin{aligned}
&\gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] - 2\|Q_i - Q^*\|_\infty && (2.133)\\
\geq&\gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right] - 2\gamma^i\|Q_0 - Q^*\|_\infty && (2.134)
\end{aligned}$$

TODO: finire dimostrazione ■

We can characterize the quality of a policy obtained at the i-th iteration, if we want an ϵ-error we have to solve:

$$\frac{2\gamma^i}{1-\gamma}\|Q_0 - Q^*\|_\infty \leq \epsilon \quad (2.135)$$

for i, to get the number of iterations that we need. Without entering in the details of the proof, the correct number of iterations should be

$$i \geq \frac{1}{1-\gamma}\log\left(\frac{2}{\epsilon(1-\gamma)^2}\right). \quad (2.136)$$

2.4 Policy Iteration

In the previous section we have described an algorithm that iterates over the value functions (or the Q-functions) and provides in the end a Q-function that is approximately the optimal Q-function, from this we can define an approximation of the optimal policy π^*. In this section we describe an algorithm that generates a policy at each iteration: $\{\pi_0, \pi_1, \ldots, \pi_T\}$. The logic of the algorithm is simple:

- We start with an initial guess π_0.
- We start iterating, at each iteration we evaluate the Q-function for our guess.
- We use the Q-function to improve the guess for the next iteration.

Algorithm 5 Policy Iteration

```
initialize $\pi_0$ with a random guess
do
    compute $Q^{\pi_t}$ for each pair $(s, a)$
    perform the policy improvement:
        $\pi_{t+1}(s) = \arg\max_{a \in A} Q^{\pi_t}(s, a), \quad \forall s \in S$    (2.137)
while convergence is not met
```

Proposition 2.4.1 : Monotonic Improvement

For two consecutive steps in Algorithm 5, the performance progress is monotonic:

$$Q^{\pi_{t+1}} \geq Q^{\pi_t} \quad \forall s, a \tag{2.138}$$

$$\|V^{\pi_t} - V^*\|_\infty \leq \gamma^t \|V^{\pi_0} - V^*\|_\infty. \tag{2.139}$$

Proof: We want to prove that $Q^{\pi_{t+1}} \geq Q^{\pi_t}$ (the policy get better, its state-action value increase). The following reasoning holds for each pair (s, a). Consider:

$$Q^{\pi_{t+1}}(s, a) - Q^{\pi_t}(s, a) \tag{2.140}$$

expand the definitions and simplify $r(s, a)$:

$$Q^{\pi_{t+1}}(s, a) - Q^{\pi_t}(s, a) = \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} \left[Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_t(s')) \right] \tag{2.141}$$

we add and subtract $Q^{\pi_t}(s', \pi_{t+1}(s'))$:

$$\gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} \left[Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_t(s')) \right] \tag{2.142}$$

$$= \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} \left[Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_{t+1}(s')) + Q^{\pi_t}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_t(s')) \right] \tag{2.143}$$

$$= \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} [Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_{t+1}(s')) \underbrace{+ Q^{\pi_t}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_t(s'))}_{\geq 0}] \tag{2.144}$$

$$\geq \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} [Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_{t+1}(s'))] \tag{2.145}$$

We have obtained the following inequality:

$$Q^{\pi_{t+1}}(s, a) - Q^{\pi_t}(s, a) \geq \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} [Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_{t+1}(s'))] \tag{2.146}$$

The term within the expected value has the exact same structure as the left-handed difference, but evaluated at the next time step (state s', action $\pi_{t+1}(s')$). By repeatedly applying the same reasoning (expansion and policy improvement) to the subsequent terms, we will accumulate increasing powers of the discount factor γ:

$$\geq \gamma^2 \mathbb{E}[\ldots] \geq \gamma^3 \mathbb{E}[\ldots] \cdots \geq \lim_{k \to \infty} \gamma^k \mathbb{E}[\ldots] \tag{2.147}$$

Since the discount factor is strictly less than 1 and the rewards (and therefore the Q-values) are assumed to be bounded, the limit of γ^k for $k \to \infty$ converges to 0, hence:

$$Q^{\pi_{t+1}}(s, a) - Q^{\pi_t}(s, a) \geq 0 \implies Q^{\pi_{t+1}}(s, a) \geq Q^{\pi_t}(s, a). \tag{2.148}$$

■

Usually we apply policy iteration until two consecutive policies are identical for any state.

2.4.1 Performance Difference

TODO: L05

2.5 Finite-Horizon MDPs

We consider finite-horizon MDPs, by setting $H \geq 0$ as the horizon: every trajectory of the system will be at most long H, an MDP of that kind is a 6-tuple:

$$(S, A, r, T, H, \mu_0) \tag{2.149}$$

with μ_0 being the initial state distribution ($s_0 \sim \mu_0$). Theoretically we don't need to include a discount factor, we can compute the expected value by summing over all the next states up to a finite number, practically, a discount factor is considered also for these MDPs.

We will consider time-dependent policies:

$$\pi = \{\pi_0, \pi_1, \pi_2, \ldots, \pi_{H-1}\} \tag{2.150}$$

hence, a policy π assign at each pair state-time step an action:

$$\pi(s, t) = \pi_t(s). \tag{2.151}$$

For example, for the initial time steps π might explore, then with t growing, it might start to exploit (this kind of strategy is very common in control). Let's discuss about policy iteration for a finite-horizon MDP with time varying policy. In one execution, the first state is sampled $s_0 \sim \mu_0$, and the first action is

$$a_0 = \pi_0(s_0) \tag{2.152}$$

the transition function is stochastic, hence s_1 is given by:

$$s_1 \sim p(\cdot \mid s_0, a_0) \tag{2.153}$$

with p being the transition distribution. Then we select:

$$a_1 = \pi_1(s_1) \tag{2.154}$$

the first elements of the trajectory are:

$$(s_0, a_0, s_1, a_1) \tag{2.155}$$

the trajectory goes on up to $H - 1$:

$$(s_0, a_0, s_1, a_1, s_2, a_2, \ldots, s_{H-1}, a_{H-1}). \tag{2.156}$$

The finite-horizon value function is defined as

$$V_h^\pi(s) = \mathbb{E}_\pi \left[r(s, \pi(s)) + \sum_{\tau=h+1}^{H-1} r(s_\tau, \pi(s_\tau)) \right] \tag{2.157}$$

where $s_{\tau+1} \sim p(\cdot \mid s_\tau, \pi(s_\tau))$. The state-action function is

$$Q_h^\pi(s, a) = \mathbb{E}_\pi \left[r(s, a) + \sum_{\tau=h+1}^{H-1} r(s_\tau, \pi(s_\tau)) \right] \tag{2.158}$$

where $s_{\tau+1} \sim p(\cdot \mid s_\tau, \pi(s_\tau))$. We want to find an optimal policy for each time step:

$$\pi^* = \{\pi_0^*, \pi_1^*, \pi_2^*, \ldots, \pi_{H-1}^*\} \tag{2.159}$$

Observation 2.5.1 : B

fore the final step, the action-value function is equal to the reward, since there will be no other states (the horizon is H).

$$Q_{H-1}^*(s, a) = r(s, a). \tag{2.160}$$

This implies that we can easily infer what is the best action on s at the time step $H - 1$:

$$\pi_{H-1}^*(s) = \arg\max_a r(s, a) \tag{2.161}$$

so we can infer the value:

$$V^*_{H-1}(s) = Q^*_{H-1}(s, \pi^*_{H-1}(s)). \tag{2.162}$$

Thanks to the Bellman equation we can compute the state-action value for $H-2$:

$$Q^*_{H-2}(s,a) = r(s,a) + V^*_{H+1}(s) \tag{2.163}$$

by reasoning backwards in this way we can compute the optimal policy for all the time steps $h = H-1, H-2, \ldots, 1, 0$.

CHAPTER

3

CONTROL THEORY AND REINFORCEMENT LEARNING

In this chapter we briefly discuss the bridge between optimal control and reinforcement learning. In control theory we denote by $\mathbf{x} \in \mathbb{R}^d$ the state and by $\mathbf{u} \in \mathbb{R}^k$ the input (action) that we can perform. The goal is to find a control policy π, which selects the input $\mathbf{u}$ at each time step, that minimizes the integral over time (the sum, in discrete time) of a given cost function $c(\mathbf{x}, \mathbf{u})$.

In our domain the state $\mathbf{x}$ is denoted s, while the control input $\mathbf{u}$ is denoted a. In this chapter we keep the notation of control theory, and we remark that, differently from the previous chapters, state and input are **continuous** vectors and not elements of a discrete set. In RL we want to maximize the reward, which is equivalent to minimizing a cost function (the cost is the opposite of the reward). In our context, we formalize an optimal control problem as follows

$$\min_{\pi} \quad c_H(\mathbf{x}_H) + \sum_{h=0}^{H-1} c_h(\mathbf{x}_h, \pi_h(\mathbf{x}_h)) \tag{3.1}$$

$$\text{where } \mathbf{x}_0 \sim \mu_0 \tag{3.2}$$

the state trajectory that is implicit here is

$$(\mathbf{x}_0, \mathbf{u}_0, \ldots, \mathbf{x}_h, \underbrace{\mathbf{u}_h}_{\pi_h(\mathbf{x}_h)}, \ldots, \mathbf{x}_{H-1}, \mathbf{u}_{H-1}, \mathbf{x}_H) \tag{3.3}$$

that is defined by the transition dynamics

$$\mathbf{x}_{h+1} = f(\mathbf{x}_h, \mathbf{u}_h) \tag{3.4}$$

furthermore:

- c_h is the cost function at the time step h, hence we are assuming that the cost might depend not only on the pair state-action, but also on the time.
- c_H is the cost function at the final step, it does not depend on the input since the execution ends here.

We take into account that the transition dynamics f, the cost function or the policy might be stochastic, so we consider the expected value:

$$\min_{\pi} \quad \mathbb{E}\left[c_H(\mathbf{x}_H) + \sum_{h=0}^{H-1} c_h(\mathbf{x}_h, \pi_h(\mathbf{x}_h))\right] \tag{3.5}$$

$$\text{where } \mathbf{x}_0 \sim \mu_0. \tag{3.6}$$

3.1 Linear Quadratic Regulator

We will show how the problem of finding an optimal policy can be solved with LQR. We will give a general discussion about linear systems with quadratic cost, and then show how it fits the optimal policy search. A discrete-time linear system is defined as follows:

$$\mathbf{x}_{t+1} = F_x \mathbf{x}_t + F_u \mathbf{u}_t \tag{3.7}$$

with F_x being a $d \times d$ matrix and F_u a $d \times k$ matrix. The dynamics is linear both in the state and in the control input, this is our transition function. We will consider only **quadratic** cost functions:

$$c(\mathbf{x}, \mathbf{u}) = \mathbf{x}^\top C_x \mathbf{x} + \mathbf{u}^\top C_u \mathbf{u} \tag{3.8}$$

with C_x being a positive semidefinite matrix and C_u a positive definite matrix. Let V_h^* be the value function, with $h < H$ the current time step, i.e. the minimum cost that can be obtained from the state $\mathbf{x}$ at time h onwards (the **cost-to-go**). We define the state-action value by using the cost instead of the reward:

$$Q_h^*(\mathbf{x}, \mathbf{u}) = c(\mathbf{x}, \mathbf{u}) + \mathbb{E}_{\mathbf{x}' \sim p(\cdot|\mathbf{x},\mathbf{u})} \left[V_{h+1}^*(\mathbf{x}') \right] \tag{3.9}$$

by expanding the cost we get:

$$Q_h^*(\mathbf{x}, \mathbf{u}) = \mathbf{x}^\top C_x \mathbf{x} + \mathbf{u}^\top C_u \mathbf{u} + \mathbb{E}_{\mathbf{x}' \sim p(\cdot|\mathbf{x},\mathbf{u})} \left[V_{h+1}^*(\mathbf{x}') \right] \tag{3.10}$$

we have the assumption (for now) that the dynamics is deterministic, defined in Equation (3.7), so:

$$\begin{aligned} Q_h^*(\mathbf{x}, \mathbf{u}) &= \mathbf{x}^\top C_x \mathbf{x} + \mathbf{u}^\top C_u \mathbf{u} + V_{h+1}^*(\mathbf{x}') && (3.11) \\ &= \mathbf{x}^\top C_x \mathbf{x} + \mathbf{u}^\top C_u \mathbf{u} + V_{h+1}^*(F_x \mathbf{x} + F_u \mathbf{u}). && (3.12) \end{aligned}$$

Now we make a strong assumption:

The value function V_h^* is a **quadratic form**.

So, we can write $V_h^*(\mathbf{x}) = \mathbf{x}^\top P_h \mathbf{x}$, with P_h being a symmetric positive semidefinite matrix. We rewrite the action-value function as:

$$\begin{aligned} Q_h^*(\mathbf{x}, \mathbf{u}) &= \mathbf{x}^\top C_x \mathbf{x} + \mathbf{u}^\top C_u \mathbf{u} + V_{h+1}^*(F_x \mathbf{x} + F_u \mathbf{u}) && (3.13) \\ &= \mathbf{x}^\top C_x \mathbf{x} + \mathbf{u}^\top C_u \mathbf{u} + (F_x \mathbf{x} + F_u \mathbf{u})^\top P_{h+1} (F_x \mathbf{x} + F_u \mathbf{u}) && (3.14) \end{aligned}$$

we don't know yet how P_h is constructed, we are just assuming that the value is a quadratic form. Now that we have an analytic formulation of $Q_h^*(\mathbf{x}, \mathbf{u})$, we know that we can express the best policy (at the time step h) as:

$$\pi_h^*(\mathbf{x}) = \arg\min_{\mathbf{u}} Q_h^*(\mathbf{x}, \mathbf{u}) \tag{3.15}$$

notice how we are minimizing and not maximizing since we have a cost instead of a reward (but the two formulations are equivalent). Since $C_u + F_u^\top P_{h+1} F_u$ is positive definite, $Q_h^*(\mathbf{x}, \cdot)$ is strictly convex in $\mathbf{u}$, hence to find its minimum we can set the gradient to zero:

$$\nabla_u Q_h^*(\mathbf{x}, \mathbf{u}) = \mathbf{0}. \tag{3.16}$$

The gradient is:

$$\nabla_u Q_h^*(\mathbf{x}, \mathbf{u}) = 2C_u \mathbf{u} + 2F_u^\top P_{h+1}(F_x \mathbf{x} + F_u \mathbf{u}) \tag{3.17}$$

now we solve Equation (3.16) for $\mathbf{u}$:

$$\begin{aligned} 2C_u \mathbf{u} + 2F_u^\top P_{h+1}(F_x \mathbf{x} + F_u \mathbf{u}) &= \mathbf{0} && (3.18) \\ \implies \mathbf{u} = -(C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x \mathbf{x} &= K_h^* \mathbf{x} && (3.19) \end{aligned}$$

we have denoted K_h^* that matrix, in this way we have found the optimal policy at time h:

$$\pi_h^*(\mathbf{x}) = K_h^* \mathbf{x} \tag{3.20}$$

Now we can replace $\mathbf{u} = K_h^*\mathbf{x}$ in Equation (3.14), by definition, this is exactly the value function:

$$Q_h^*(\mathbf{x}, \pi_h^*(\mathbf{x})) = V_h^*(\mathbf{x}) = \mathbf{x}^\top C_x \mathbf{x} + (K_h^*\mathbf{x})^\top C_u (K_h^*\mathbf{x}) + (F_x\mathbf{x} + F_u K_h^*\mathbf{x})^\top P_{h+1}(F_x\mathbf{x} + F_u K_h^*\mathbf{x}) \tag{3.21}$$

$$= \mathbf{x}^\top (C_x + (K_h^*)^\top C_u K_h^* + (F_x + F_u K_h^*)^\top P_{h+1}(F_x + F_u K_h^*))\mathbf{x} \tag{3.22}$$

$$= \mathbf{x}^\top P_h \mathbf{x} \tag{3.23}$$

and is exactly a quadratic form ($\mathbf{x}^\top P_h \mathbf{x}$), so the assumption made at the beginning was correct. Notice how P_h can be computed when P_{h+1} is known: due to the finite horizon, we are able to compute in closed form the cost at the final step, hence P_H can be initialized according to the final cost c_H (for instance $P_H = C_x$ if $c_H(\mathbf{x}) = \mathbf{x}^\top C_x \mathbf{x}$, or $P_H = 0$ if there is no final cost), then we can reconstruct P_h for all h backwards.

Algorithm 6 Value Iteration as LQR

```
initialize $P_H$
for $h = H-1, H-2, \dots, 0$ (backwards) do
    Set $K_h^* = -(C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x$
    Compute the best action $\mathbf{u} = \pi_h^*(\mathbf{x}) = K_h^*\mathbf{x}$
    Set $P_h = C_x + (K_h^*)^\top C_u K_h^* + (F_x + F_u K_h^*)^\top P_{h+1}(F_x + F_u K_h^*)$
    Compute the value $V_h^*(\mathbf{x}) = \mathbf{x}^\top P_h \mathbf{x}$
end for
```

Notice how the best action at a given state is a term proportional to that state, in fact this is a *proportional regulator*. Here $V_h^*(\mathbf{x}) = \mathbf{x}^\top P_h \mathbf{x}$ is known as the **cost-to-go**.

3.1.1 Extensions

The first assumption that we drop is that the system is time invariant: consider dynamics that depend also on the time t:

$$\mathbf{x}_{t+1} = F_{x,t}\mathbf{x}_t + F_{u,t}\mathbf{u}_t \tag{3.24}$$

with a cost that also depends on t:

$$c_t(\mathbf{x}_t, \mathbf{u}_t) = \mathbf{x}_t^\top C_{x,t}\mathbf{x}_t + \mathbf{u}_t^\top C_{u,t}\mathbf{u}_t. \tag{3.25}$$

Algorithm 6 can be performed in the same way; the matrices simply depend on the time.

Let us switch back to time invariant systems, and consider a system that is not linear but *affine*:

$$\mathbf{x}_{t+1} = F_x\mathbf{x}_t + F_u\mathbf{u}_t + \mathbf{c} \tag{3.26}$$

with $\mathbf{c}$ being a constant vector. We can still apply Algorithm 6 by transforming the system into a linear system through the change of coordinates $\mathbf{z} = \begin{pmatrix}\mathbf{x}^\top & 1\end{pmatrix}^\top$:

$$\mathbf{z}_{t+1} = \begin{pmatrix}\mathbf{x}_{t+1} \\ 1\end{pmatrix} = \begin{pmatrix}F_x & \mathbf{c} \\ \mathbf{0}^\top & 1\end{pmatrix}\begin{pmatrix}\mathbf{x}_t \\ 1\end{pmatrix} + \begin{pmatrix}F_u \\ \mathbf{0}^\top\end{pmatrix}\mathbf{u}_t \tag{3.27}$$

$$= F'_x\mathbf{z}_t + F'_u\mathbf{u}_t. \tag{3.28}$$

The cost is still quadratic in the new coordinates, since $\mathbf{x}^\top C_x \mathbf{x} = \mathbf{z}^\top \begin{pmatrix}C_x & \mathbf{0} \\ \mathbf{0}^\top & 0\end{pmatrix}\mathbf{z}$.

Now we take into account *stochastic systems*; in particular, we consider noisy dynamics that are perturbed by a zero-mean Gaussian noise:

$$\mathbf{x}_{t+1} = F_x\mathbf{x}_t + F_u\mathbf{u}_t + \mathbf{w}_t \tag{3.29}$$

$$\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 I). \tag{3.30}$$

In this case, the optimal policy is the same, but the cost-to-go has an additional term that depends on the variance of the noise and that cannot be controlled:

$$V_h^*(\mathbf{x}) = \mathbf{x}^\top P_h \mathbf{x} + p_h \tag{3.31}$$

where

$$p_h = \operatorname{tr}\left(\sigma^2 P_{h+1}\right) + p_{h+1} \tag{3.32}$$

$$p_H = 0. \tag{3.33}$$

There is also a formulation that is more suitable for some cases, where we apply a coordinate and input transformation in order to have a cost function that describes the cost of a certain control input and also the cost of *changing* that control input. Starting from a standard system:

$$\mathbf{x}_{t+1} = F_x \mathbf{x}_t + F_u \mathbf{u}_t, \tag{3.34}$$

we consider the following equivalent system:

$$\begin{pmatrix} \mathbf{x}_{t+1} \\ \mathbf{u}_t \end{pmatrix} = \begin{pmatrix} F_x & F_u \\ \mathcal{O} & I \end{pmatrix} \begin{pmatrix} \mathbf{x}_t \\ \mathbf{u}_{t-1} \end{pmatrix} + \begin{pmatrix} F_u \\ I \end{pmatrix} \Delta \mathbf{u}_t \tag{3.35}$$

where $\mathcal{O}$ is the zero matrix and $\Delta \mathbf{u}_t = \mathbf{u}_t - \mathbf{u}_{t-1}$. We rewrite the system as

$$\mathbf{x}'_{t+1} = F'_x \mathbf{x}'_t + F'_u \Delta \mathbf{u}_t. \tag{3.36}$$

$\Delta \mathbf{u}_t$ is the new input, and is the change in controls. The new cost is

$$(\mathbf{x}')^\top \begin{pmatrix} C_x & \mathcal{O} \\ \mathcal{O} & C_u \end{pmatrix} \mathbf{x}' + \Delta \mathbf{u}^\top C'_u \Delta \mathbf{u}, \tag{3.37}$$

where C'_u is the matrix that penalizes the change in controls.

3.2 LQR for Non-Linear Systems

Now we consider a general non-linear system with the dynamics expressed through a non-linear function f:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t). \tag{3.38}$$

Let $\mathbf{x}^*$ be a forced equilibrium:

$$f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{x}^* \tag{3.39}$$

with $\mathbf{u}^*$ being the required input. We linearize the system around that equilibrium using a first order Taylor expansion:

$$\mathbf{x}_{t+1} \approx f(\mathbf{x}^*, \mathbf{u}^*) + \nabla_x f(\mathbf{x}_t - \mathbf{x}^*) + \nabla_u f(\mathbf{u}_t - \mathbf{u}^*) \tag{3.40}$$

with $\nabla_x f$ and $\nabla_u f$ being the Jacobian matrices (with respect to $\mathbf{x}$ and $\mathbf{u}$) evaluated at the equilibrium $(\mathbf{x}^*, \mathbf{u}^*)$. We denote by A and B the Jacobian matrices, by $\mathbf{z}_t = \mathbf{x}_t - \mathbf{x}^*$ the variation in the state with respect to the equilibrium and by $\mathbf{v}_t = \mathbf{u}_t - \mathbf{u}^*$ the variation in the input:

$$\mathbf{x}_{t+1} \approx \underbrace{f(\mathbf{x}^*, \mathbf{u}^*)}_{\mathbf{x}^*} + A(\mathbf{x}_t - \mathbf{x}^*) + B(\mathbf{u}_t - \mathbf{u}^*) \implies \tag{3.41}$$

$$\mathbf{x}_{t+1} - \mathbf{x}^* \approx A(\mathbf{x}_t - \mathbf{x}^*) + B(\mathbf{u}_t - \mathbf{u}^*) \implies \tag{3.42}$$

$$\mathbf{z}_{t+1} \approx A\mathbf{z}_t + B\mathbf{v}_t. \tag{3.43}$$

We have a linear system; clearly, to reach the equilibrium we should bring the new state $\mathbf{z}$ to zero. The usual way to define the control action is with a proportional controller:

$$\mathbf{v}_t = K\mathbf{z}_t \implies \mathbf{u}_t = \mathbf{u}^* + K(\mathbf{x}_t - \mathbf{x}^*). \tag{3.44}$$

If the cost is quadratic, we can apply the procedure shown in Algorithm 6; if not, it can be approximated with a quadratic function through a second order Taylor expansion.

3.2.1 Trajectory Tracking

Now we want to define a controller that performs trajectory tracking while minimizing the cost function. The desired trajectory is

$$\mathbf{x}_0^*, \mathbf{x}_1^*, \ldots, \mathbf{x}_H^*. \tag{3.45}$$

Each single transition of the trajectory is a forced equilibrium

$$f(\mathbf{x}_t^*, \mathbf{u}_t^*) = \mathbf{x}_{t+1}^*. \tag{3.46}$$

In other words, we are trying to find a control input for each time step

$$\mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{H-1} \tag{3.47}$$

that minimizes the overall cost:

$$\min_{\mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{H-1}} \quad \sum_{t=0}^{H-1} (\mathbf{x}_t - \mathbf{x}_t^*)^\top C_x (\mathbf{x}_t - \mathbf{x}_t^*) + (\mathbf{u}_t - \mathbf{u}_t^*)^\top C_u (\mathbf{u}_t - \mathbf{u}_t^*) \tag{3.48}$$

$$\text{s.t.} \quad \mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t). \tag{3.49}$$

The solution is simple: at each time step we linearize the system around the desired trajectory, and we treat it as a linear time-varying system:

$$\mathbf{x}_{t+1} - \mathbf{x}_{t+1}^* \approx A_t(\mathbf{x}_t - \mathbf{x}_t^*) + B_t(\mathbf{u}_t - \mathbf{u}_t^*) \tag{3.50}$$

$$\mathbf{u}_t - \mathbf{u}_t^* = K_t(\mathbf{x}_t - \mathbf{x}_t^*). \tag{3.51}$$

3.3 Iterative LQR

Now we try to solve the most general case of a discrete-time optimal control problem:

$$\min_{\mathbf{u}_0, \ldots, \mathbf{u}_{H-1}} \quad g_H(\mathbf{x}_H) + \sum_{h=0}^{H-1} g_h(\mathbf{x}_h, \mathbf{u}_h) \tag{3.52}$$

$$\text{s.t.} \quad \mathbf{x}_{h+1} = f(\mathbf{x}_h, \mathbf{u}_h). \tag{3.53}$$

A problem of this kind can be solved with Algorithm 7.

Algorithm 7 Iterative LQR

```
Initialize a control policy $\pi^{(0)}$.
for $i = 0, 1, \ldots, I$ do
    Execute $\pi^{(i)}$ and collect the state-input trajectory $\{\mathbf{x}_t^{(i)}, \mathbf{u}_t^{(i)}\}$ for $t = 0, \ldots, H$.
    Obtain an LQ approximation of this problem by linearizing around $\{\mathbf{x}_t^{(i)}, \mathbf{u}_t^{(i)}\}$ with first order Taylor expansion for the dynamics and second order Taylor expansion for the cost.
    Compute $\pi^{(i+1)}$ with the LQR formulation for linear time variant systems.
end for
```